

JADAVPUR UNIVERSITY

NAME : DIPTARKO BHATTACHARJEE

CLASS: BCSE-UG2-SEM1-A1-2024-28

ROLL : 002410501013

SUB : DSA LAB REPORT

Assignment - 1:

1. Define an ADT List, L with N elements, which are stored in a file. The associated operations are given below. Write a menu driven program for the same using (i) Array and (ii) Linked list.
a) Create (L): Create a list of N elements.
b) Length (L): Returns the length of the list L, N.
c) PrintList (L): Print the elements of L in order of occurrence left-to-right (or right-to- left).
d) Retrieve (i, L): Retrieve the ith element of L.
e) Store (x, i, L): Store a new value, x at ith position in L.
f) Insert (x, i, L): Insert a new element at ith position of L, moving the elements at i and the following positions to the next higher position.
g) Delete (i, L): Delete the ith element from L, moving the elements of the following positions to the next lower position.
h) Search (x, L): Search the key x in the list L. If found, return its position in the list, otherwise return (-1) indicating not found.

Code: - (using Array)

```
#include <stdio.h>

#define MAX 100

typedef struct {
   int arr[MAX];
   int n;
} array;

int status = 0;

void Create(array *list){
   FILE *fp;
```

```
   fp = fopen("List.txt", "r");
   list->n = 0;
   while (fscanf(fp, "%d", &list->arr[list->n]) == 1){
       list->n++;
   }
   fclose(fp);
}

int Length(array *list){
   if (list->n==0){
       status=1;
       return 0;
   }
   return list->n;
}

void PrintList(array *list){
   printf("Enter how you want to display the elements:\n1 - Left-to-right\n2 -
Right-to-left\n");
   printf("Enter your choice: ");
   int choice;
   scanf("%d", &choice);
   if (choice==1){

       for (int i=0;i<list->n;i++){
           printf("%d ", list->arr[i]);
       }
   }
   else {
       for (int i=list->n-1;i>=0;i--){
           printf("%d ", list->arr[i]);
       }
   }
   printf("\n");
   return;
}

int Retrieve(int i, array *list){
   return list->arr[i-1];
```

```
}

void Store(int x, int i, array *list){
   list->arr[i-1] = x;
}

void Insert(int x, int i, array *list){
   i = i-1;
   for (int j=list->n-1;j>=i;j--){
       list->arr[j+1] = list->arr[j];
   }
   list->arr[i] = x;
   list->n++;
   return;
}

void Delete(int i, array *list){
   i = i-1;
   for (int j=i+1;j<list->n;j++){
       list->arr[j-1] = list->arr[j];
   }
   list->n--;
   return;
}

int Search(int x, array *list){
   for (int i=0;i<list->n;i++){
       if (list->arr[i]==x){
           return i+1;
       }
   }
   return -1;
}

int main(){
   printf("A new list is going to be created, containing elements of the file
'Lists.txt'.....\n");
   array list;
   Create(&list);
```

```
   while (27){

       printf("Enter the desired operation to be done on the list:\n");
       printf("1 - Find the length of the list\n2 - Print the elements of the
list\n3 - Retrieve an element from the list\n");
       printf("4 - Store a new value at a position in the list\n5 - Insert an
element into the list\n");
       printf("6 - Delete an element from the list\n7 - Search for an element
in the list\n0 - Exit\n");
       printf("Enter your choice: ");

       int choice;
       scanf("%d", &choice);

       switch (choice){

           case 1: {
               int length = Length(&list);
               if (status == 1){
                   status = 0;
                   break;
               }
               printf("The present length of the list is: %d\n", length);
               break;
           }

           case 2: {
               PrintList(&list);
               break;
           }

           case 3: {
               printf("Enter the position of the element that you want to
retrieve: ");
               int pos;
               scanf("%d", &pos);
               int ele = Retrieve(pos, &list);
               if (status == 1){
```

```
                status = 0;
                break;
            }
            printf("The element at the %dth position of the list is: %d\n",
pos, ele);
            break;
        }

        case 4: {
            printf("Enter the position of the list where you want to store a
new element: ");
            int pos;
            scanf("%d", &pos);
            printf("Enter the new element that will update the %dth element
in the list: ", pos);
            int ele;
            scanf("%d", &ele);
            Store(ele, pos, &list);
            break;
        }

        case 5: {
            printf("Enter the position of the list where you want to insert
a new element: ");
            int pos;
            scanf("%d", &pos);
            printf("Enter the new element that will be inserted at the %dth
position of the list: ", pos);
            int ele;
            scanf("%d", &ele);
            Insert(ele, pos, &list);
            break;
        }

        case 6: {
            printf("Enter the position of the element which needs to be
deleted: ");
            int pos;
            scanf("%d", &pos);
```

```
                Delete(pos, &list);
                break;
            }

            case 7: {
                printf("Enter the element whose position needs to be searched:
");
                int ele;
                scanf("%d", &ele);
                int pos = Search(ele, &list);
                if (pos == -1){
                    printf("The desired element does not exist in the list!\n");
                    break;
                }
                printf("The first occurrence of the element %d presently in the
list is: %d\n", ele, pos);
                break;
            }

            case 0: {
                return 0;
            }

            default: {
                printf("Please enter a valid choice!\n");
                break;
            }

        }
    }

    return 0;
}
```

Code: - (Using Linked List)

```
#include <stdio.h>
#include <stdlib.h>

typedef struct link{
   int num;
   struct link *next;
} link;

link *head = NULL;
int status = 0;

void createNode(int x, link *list){
   link *newNode = (link *)malloc(sizeof(link));
   newNode->num = x;
   newNode->next = NULL;
   if (head == NULL){
       head = newNode;
   }
   else {
       link *it = head;
       while (it->next != NULL){
           it = it->next;
       }
       it->next = newNode;
   }
}

void Create(link *list){
   FILE *fp = fopen("List.txt", "r");
   int ele;
   while (fscanf(fp, "%d", &ele) == 1){
       createNode(ele, list);
   }
   fclose(fp);
   printf("A new list has been successfully created!\n");
}
```

```
int Length(link *list){
   link *it = head;
   int cnt = 0;
   while (it != NULL){
       it = it->next;
       cnt++;
   }
   if (cnt == 0){
       status = 1;
       printf("No elements in list presently!\n");
   }
   return cnt;
}

void PrintList(link *list){
   link *it = head;
   int cnt = 1;
   if (head == NULL){
       printf("No elements in list presently!\n");
       return;
   }
   printf("Enter in which order the elements of the list is to be displayed:\n1
- Left-to-right\n2 - Right to left\n");
   printf("Enter your choice: ");
   int choice;
   scanf("%d", &choice);
   if (choice == 1){
       printf("The elements of the list presently are as follows:\n");
       while (it != NULL){
           printf("Element in the %dth node is: %d\n", cnt, it->num);
           it = it->next;
           cnt++;
       }
       printf("The present elements of the list were successfully printed!\n");
       return;
   }
   else {
       it = head;
       cnt = Length(list);
```

```
        int arr[cnt];
        for (int i=0;i<cnt;i++){
            arr[i] = it->num;
            it = it->next;
        }
        printf("The elements of the list presently are as follows:\n");
        for (int i=cnt-1;i>=0;i--){
            printf("Element in the %dth node is: %d\n", i+1, arr[i]);
        }
        printf("The present elements of the list were successfully printed!\n");
        return;
    }
}

int Retrieve(int i, link *list){
    if (head == NULL){
        printf("No elements in the list presently!\n");
        status = 1;
        return 0;
    }
    link *p = head;
    int cnt = 1;
    while(p->next!=NULL && cnt!=i){
        p = p->next;
        cnt++;
    }
    if (i>cnt){
        printf("List ends before given position. Please input a position between
1 and %d!\n", cnt+1);
        status = 1;
        return 0;
    }
    return p->num;
}

void Store(int x, int i, link *list){
    if (head == NULL){
        printf("No elements in the list presently!\n");
        return;
```

```
    }
    link *p = head;
    int cnt = 1;
    while(p->next!=NULL && cnt!=i){
        p = p->next;
        cnt++;
    }
    if (i>cnt){
        printf("List ends before given position. Please input a position between
1 and %d!\n", cnt+1);
        status = 1;
        return;
    }
    p->num = x;
    printf("Element succesfully updated!\n");
    return;
}

void Insert(int x, int i, link *list){
    link *newRecord = malloc(sizeof(link));
    newRecord->num = x;
    int cnt = 1;
    link *p = head;
    while (p->next != NULL){
        p = p->next;
        cnt++;
    }
    if (cnt<i-1){
        printf("List ends before given position. Please input a position between
1 and %d!\n", cnt+1);
        return;
    }
    p = head;
    if (i==1){
        head = newRecord;
        newRecord->next = p;
        printf("New element inserted at the %dth position successfully!\n", i);
        return;
    }
```

```
   else if (i<=0){
       printf("The list starts at position 1. Please input a position between 1
and %d!\n", cnt+1);
       return;
   }
   else {
       cnt = 1;
       while (cnt<i-1){
           p = p->next;
           cnt++;
       }
       newRecord->next = p->next;
       p->next = newRecord;
       printf("New element inserted at the %dth position successfully!\n", i);
       return;
   }
}

void Delete(int i, link *list){
   if (head == NULL){
       printf("No elements in the list!\n");
       return;
   }
   link *p = head;
   int cnt = 1;
   while (p->next != NULL){
       p = p->next;
       cnt++;
   }
   if (cnt<i){
       printf("List ends before given position. Please input a position within
%d!\n", cnt+1);
       return;
   }
   p = head;
   if (i==1){
       head = p->next;
       free(p);
       printf("Element successfully deleted!\n");
```

```
        return;
    }
    else if (i<=0){
        printf("The list starts at position 1. Please input a position between 1
and %d!\n", cnt+1);
        return;
    }
    cnt = 1;
    while (cnt<i-1){
        p = p->next;
        cnt++;
    }
    link *q = p;
    p = p->next;
    q->next = p->next;
    free(p);
    printf("Element successfully deleted!\n");
    return;
}

int Search(int x, link *list){
    link *p = head;
    int cnt = 1;
    while (p!=NULL && p->num!=x){
        p = p->next;
        cnt++;
    }
    if (p == NULL){
        return -1;
    }
    return cnt;
}

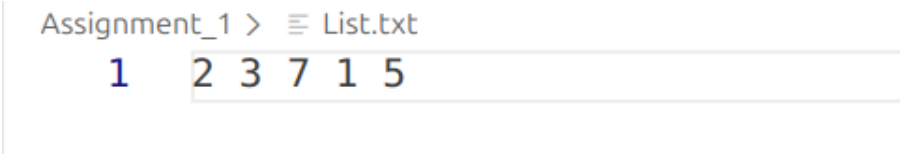
int main(){
    printf("A new list is going to be created, containing elements of the file
'Lists.txt'.....\n");
    link list;
    Create(&list);
    while (27){
```

```
       printf("Enter the desired operation to be done on the list:\n");
       printf("1 - Find the length of the list\n2 - Print the elements of the
list\n3 - Retrieve an element from the list\n");
       printf("4 - Store a new value at a position in the list\n5 - Insert an
element into the list\n");
       printf("6 - Delete an element from the list\n7 - Search for an element
in the list\n0 - Exit\n");
       printf("Enter your choice: ");
       int choice;
       scanf("%d", &choice);
       switch (choice){
           case 1: {
               int length = Length(&list);
               if (status == 1){
                   status = 0;
                   break;
               }
               printf("The present length of the list is: %d\n", length);
               break;
           }
           case 2: {
               PrintList(&list);
               break;
           }
           case 3: {
               printf("Enter the position of the element that you want to
retrieve: ");
               int pos;
               scanf("%d", &pos);
               int ele = Retrieve(pos, &list);
               if (status == 1){
                   status = 0;
                   break;
               }
               printf("The element at the %dth position of the list is: %d\n",
pos, ele);
               break;
           }
           case 4: {
```

```
                printf("Enter the position of the list where you want to store a
new element: ");
                int pos;
                scanf("%d", &pos);
                printf("Enter the new element that will update the %dth element
in the list: ", pos);
                int ele;
                scanf("%d", &ele);
                Store(ele, pos, &list);
                break;
            }
            case 5: {
                printf("Enter the position of the list where you want to insert
a new element: ");
                int pos;
                scanf("%d", &pos);
                printf("Enter the new element that will be inserted at the %dth
position of the list: ", pos);
                int ele;
                scanf("%d", &ele);
                Insert(ele, pos, &list);
                break;
            }
            case 6: {
                printf("Enter the position of the element which needs to be
deleted: ");
                int pos;
                scanf("%d", &pos);
                Delete(pos, &list);
                break;
            }
            case 7: {
                printf("Enter the element whose position needs to be searched:
");
                int ele;
                scanf("%d", &ele);
                int pos = Search(ele, &list);
                if (pos == -1){
                    printf("The desired element does not exist in the list!\n");
```

```
                    break;
                }
                printf("The first occurance of the element %d presently in the
list is: %d\n", ele, pos);
                break;
            }
            case 0: {
                printf("Thank you!\n");
                return 0;
            }
            default: {
                printf("Please enter a valid choice!\n");
                break;
            }
        }
    }
    return 0;
}
```

Output: -



```
A new list is going to be created, containing elements of the file 'Lists.txt'.....
Enter the desired operation to be done on the list:
1 - Find the length of the list
2 - Print the elements of the list
3 - Retrieve an element from the list
4 - Store a new value at a position in the list
5 - Insert an element into the list
6 - Delete an element from the list
7 - Search for an element in the list
0 - Exit
Enter your choice: 1
The present length of the list is: 5
Enter the desired operation to be done on the list:
1 - Find the length of the list
2 - Print the elements of the list
3 - Retrieve an element from the list
4 - Store a new value at a position in the list
5 - Insert an element into the list
6 - Delete an element from the list
7 - Search for an element in the list
0 - Exit
Enter your choice: 2
Enter how you want to display the elements:
1 - Left-to-right
2 - Right-to-left
Enter your choice: 1
2 3 7 1 5
```

```
Enter the desired operation to be done on the list:
1 - Find the length of the list
2 - Print the elements of the list
3 - Retrieve an element from the list
4 - Store a new value at a position in the list
5 - Insert an element into the list
6 - Delete an element from the list
7 - Search for an element in the list
0 - Exit
Enter your choice: 2
Enter how you want to display the elements:
1 - Left-to-right
2 - Right-to-left
Enter your choice: 2
5 1 7 3 2
Enter the desired operation to be done on the list:
1 - Find the length of the list
2 - Print the elements of the list
3 - Retrieve an element from the list
4 - Store a new value at a position in the list
5 - Insert an element into the list
6 - Delete an element from the list
7 - Search for an element in the list
0 - Exit
Enter your choice: 3
Enter the position of the element that you want to retrieve: 2
The element at the 2th position of the list is: 3
```

```
Enter the desired operation to be done on the list:
1 - Find the length of the list
2 - Print the elements of the list
3 - Retrieve an element from the list
4 - Store a new value at a position in the list
5 - Insert an element into the list
6 - Delete an element from the list
7 - Search for an element in the list
0 - Exit
Enter your choice: 4
Enter the position of the list where you want to store a new element: 3
Enter the new element that will update the 3th element in the list: 4
Enter the desired operation to be done on the list:
1 - Find the length of the list
2 - Print the elements of the list
3 - Retrieve an element from the list
4 - Store a new value at a position in the list
5 - Insert an element into the list
6 - Delete an element from the list
7 - Search for an element in the list
0 - Exit
Enter your choice: 2
Enter how you want to display the elements:
1 - Left-to-right
2 - Right-to-left
Enter your choice: 1
2 3 4 1 5
```

```
Enter the desired operation to be done on the list:
1 - Find the length of the list
2 - Print the elements of the list
3 - Retrieve an element from the list
4 - Store a new value at a position in the list
5 - Insert an element into the list
6 - Delete an element from the list
7 - Search for an element in the list
0 - Exit
Enter your choice: 5
Enter the position of the list where you want to insert a new element: 4
Enter the new element that will be inserted at the 4th position of the list: 7
Enter the desired operation to be done on the list:
1 - Find the length of the list
2 - Print the elements of the list
3 - Retrieve an element from the list
4 - Store a new value at a position in the list
5 - Insert an element into the list
6 - Delete an element from the list
7 - Search for an element in the list
0 - Exit
Enter your choice: 2
Enter how you want to display the elements:
1 - Left-to-right
2 - Right-to-left
Enter your choice: 1
2 3 4 7 1 5
```

```
Enter the desired operation to be done on the list:
1 - Find the length of the list
2 - Print the elements of the list
3 - Retrieve an element from the list
4 - Store a new value at a position in the list
5 - Insert an element into the list
6 - Delete an element from the list
7 - Search for an element in the list
0 - Exit
Enter your choice: 6
Enter the position of the element which needs to be deleted: 5
Enter the desired operation to be done on the list:
1 - Find the length of the list
2 - Print the elements of the list
3 - Retrieve an element from the list
4 - Store a new value at a position in the list
5 - Insert an element into the list
6 - Delete an element from the list
7 - Search for an element in the list
0 - Exit
Enter your choice: 2
Enter how you want to display the elements:
1 - Left-to-right
2 - Right-to-left
Enter your choice: 1
2 3 4 7 5
```

```
Enter the desired operation to be done on the list:
1 - Find the length of the list
2 - Print the elements of the list
3 - Retrieve an element from the list
4 - Store a new value at a position in the list
5 - Insert an element into the list
6 - Delete an element from the list
7 - Search for an element in the list
0 - Exit
Enter your choice: 7
Enter the element whose position needs to be searched: 4
The first occurance of the element 4 presently in the list is: 3
Enter the desired operation to be done on the list:
1 - Find the length of the list
2 - Print the elements of the list
3 - Retrieve an element from the list
4 - Store a new value at a position in the list
5 - Insert an element into the list
6 - Delete an element from the list
7 - Search for an element in the list
0 - Exit
Enter your choice: 8
Please enter a valid choice!
Enter the desired operation to be done on the list:
1 - Find the length of the list
2 - Print the elements of the list
3 - Retrieve an element from the list
4 - Store a new value at a position in the list
5 - Insert an element into the list
6 - Delete an element from the list
7 - Search for an element in the list
0 - Exit
Enter your choice: 0
```

2. Given two sorted lists, $L1$ and $L2$. Write a function to compute $L1 \cap L2$ using only the basic list operations.

Code: -

```
#include <stdio.h>

#define MAX 1000

void getIntersection(int list1[], int n1, int list2[], int n2, int result[],
int *n3){
   int i=0, j=0, k=0;
   while (i<n1 && j<n2){
       if(list1[i]==list2[j]){
           result[k++] = list1[i];
           i++;
           j++;
       }
       else if(list1[i]<list2[j]){
           i++;
       }
       else {
           j++;
       }
   }
   *n3 = k;
}

int main(){
   int arr1[] = {1, 2, 3, 4, 5};
   int n1 = 5;
   int arr2[] = {3, 4, 5, 6, 7, 8};
   int n2 = 6;
   int n3 = MAX;
   int res[n3];
   getIntersection(arr1, n1, arr2, n2, res, &n3);
   for (int i=0;i<n3;i++){
       printf("%d ", res[i]);
   }
```

```
    printf("\n");
    return 0;
}
```

Output: -

```
3 4 5
```

3. Given two sorted lists, $L1$ and $L2$. Write a function to compute $L1 \cup L2$ using only the basic list operations.

Code: -

```
#include <stdio.h>

#define MAX 1000

void getUnion(int list1[], int n1, int list2[], int n2, int result[], int
*n3){
    int i=0, j=0, k=0;
    while (i<n1 && j<n2){
        if(list1[i]==list2[j]){
            result[k++] = list1[i];
            i++;
            j++;
        }
        else if(list1[i]<list2[j]){
            result[k++] = list1[i++];
        }
        else {
            result[k++] = list2[j++];
        }
    }
    while(i<n1){
        result[k++] = list1[i++];
    }
    while(j<n2){
```

```
        result[k++] = list2[j++];
    }
    *n3 = k;
}

int main(){
    int arr1[] = {1, 2, 3, 4, 5};
    int n1 = 5;
    int arr2[] = {3, 4, 5, 6, 7, 8};
    int n2 = 6;
    int n3 = MAX;
    int res[n3];
    getUnion(arr1, n1, arr2, n2, res, &n3);
    for (int i=0;i<n3;i++){
        printf("%d ", res[i]);
    }
    printf("\n");
    return 0;
}
```

Output: -

```
1 2 3 4 5 6 7 8
```

4. Write a function to add two polynomials.

Code: -

```
#include <stdio.h>
#include <stdlib.h>

int max(int a, int b) {
    return (a>b)?a:b;
}
int min(int a, int b) {
    return (a<b)?a:b;
}
```

```
int *addPolynomial(int poly1[], int poly2[], int n1, int n2){
   int size = max(n1, n2);
   int *result = malloc(size*sizeof(int));
   if (result == NULL) {
       printf("Memory allocation failed!\n");
       exit(1);
   }
   for (int i = 0; i < size; i++) {
       int coeff1 = (i < n1)?poly1[i]:0;
       int coeff2 = (i < n2) ? poly2[i]:0;
       result[i] = coeff1+coeff2;
   }
   return result;
}


int main() {
   int poly1[] = {1, 0, 0, 3, 6};
   int n1 = 5;

   int poly2[] = {0, 2, 0, 4, 3, 1};
   int n2 = 6;

   int size = max(n1, n2);
   int *res = addPolynomial(poly1, poly2, n1, n2);

   for (int i=0; i<size; i++)
       printf("%d ", res[i]);

   printf("\n");

   free(res);
   return 0;
}
```

Output: -



5. Write a program to evaluate a postfix expression.

Code: -

```
#include<stdio.h>
#include<stdlib.h>
#include<math.h>
#include<string.h>

#define MAX 1000

struct node {
   int data;
   struct node* next;
};

struct node* top  = NULL;
int stack[MAX];
char infix[MAX];
char postfix[MAX];

void push(int val){
   struct node* newnode = (struct node*)malloc(sizeof(struct node));
   if(newnode == NULL){
       printf("Stack overflow!!\n");
       return;
   }
   newnode->data = val;
   newnode->next = top;
   top = newnode;
}

int pop(){
   struct node* p = top;
   if(top == NULL){
       printf("Stack underflow!\n");
       exit(1);
   }
   int val = top->data;
```

```
   top = top->next;
   free(p);
   return val;
}

int getTop(){
   return top->data;
}

int isEmpty(){
   if(top == NULL) return 1;
   else return 0;
}

void print(){
   struct node* p = top;
   while(p!=NULL){
       printf("%d ",p->data);
       p = p->next;
   }
   printf("\n");
}

int precedence(char symbol){
   switch(symbol){
       case '^': return 3;
       case '*': return 2;
       case '/': return 2;
       case '+': return 1;
       case '-': return 1;
       default: return 0;
   }
}

void infix_postfix(){
   int i = 0, j=0;
   char symbol;
   char next;
   for(int i=0; i<strlen(infix); i++){
```

```
        symbol = infix[i];
        switch(symbol){
            case '(':
                push(symbol);
                break;
            case ')':
                while((next = pop()) != '('){
                    postfix[j] = next;
                    j++;
                }
                break;
            case '+':
            case '-':
            case '*':
            case '/':
            case '^':
                while(isEmpty() == 0 && precedence((char)getTop()) >=
precedence(symbol)){
                    postfix[j] = pop();
                    j++;
                }
                push(symbol);
                break;
            default:
                postfix[j++] = symbol;
                break;
        }
    }
    while(isEmpty() == 0){
        postfix[j++] = pop();
    }
    postfix[j] = '\0';
}

void printPostfix(){
    int i=0;
    printf("The postfix expression is:\n");
    while(postfix[i] != '\0'){
        printf("%c",postfix[i]);
```

```
        i++;
    }
    printf("\n");
}

int postfix_evaluation(){
    int a, b;
    char ch;
    for(int i=0;i<strlen(postfix);i++){
        ch = postfix[i];
        if(ch>='0' && ch<='9'){
            push(ch-'0');
        }
        else{
            a = pop();
            b = pop();
            switch(ch){
                case '+':
                    push(b+a);
                    break;
                case '-':
                    push(b-a);
                    break;
                case '*':
                    push(b*a);
                    break;
                case '/':
                    push(b/a);
                    break;
                case '^':
                    push(pow(b,a));
                    break;
            }
        }
    }
    return pop();
}
```

```
int main(){
   printf("Enter the infix expression:\n");
   fgets(infix, MAX, stdin);
   size_t len = strlen(infix);
   if (len > 0 && infix[len-1] == '\n') {
       infix[len-1] = '\0';
   }
   infix_postfix();
   printPostfix();
   int result;
   result = postfix_evaluation();
   printf("The evaluated reuslt is: %d\n",result);
   return 0;
}
```

Output:

```
Enter the infix expression:
(9+5)*4
The postfix expression is:
95+4*
The evaluated reuslt is: 56
```

6. Write a program to convert a postfix expression to infix expression.

Code: -

```
#include<stdio.h>
#include<stdlib.h>
#include<math.h>
#include<string.h>
#define MAX 100

struct node {
   char* data;
   struct node* next;
};
```

```
struct node* top  = NULL;

char infix[MAX];
char postfix[MAX];

void push(char *val){
   struct node* newnode = (struct node*)malloc(sizeof(struct node));
   if(newnode == NULL){
       printf("Stack overflow!!\n");
       return;
   }
   int string_len = strlen(val);
   char* string = (char*)malloc(sizeof(char)*string_len+1);
   strcpy(string,val);
   newnode->data = string;
   newnode->next = top;
   top = newnode;
}

char* pop(){
   struct node* p = top;
   if(top == NULL){
       printf("Stack underflow!!\n");
       exit(1);
   }
   char* val = top->data;
   top = top->next;
   free(p);
   return val;
}

char* getTop(){
   return top->data;
}

int isEmpty(){
   if(top == NULL) return 1;
   else return 0;
}
```

```
void print(){
   struct node* p = top;
   while(p!=NULL){
       printf("%s ",p->data);
       p = p->next;
   }
   printf("\n");
}

char* postfix_infix(){
   char *a, *b;
   char ch;
   for(int i=0;i<strlen(postfix);i++){
       ch = postfix[i];
       if(ch-'0'>=0 && ch-'9'<=9){
           char temp[2];
           temp[0] = ch;
           temp[1] = '\0';
           push(temp);
       }
       else{
           a = pop();
           b = pop();
           char res[MAX];
           strcpy(res,"(");
           strcat(res,b);
           char buffer[2];
           buffer[1] = '\0';
           switch(ch){
               case '+':
                   buffer[0] = '+';
                   break;
               case '-':
                   buffer[0] = '-';
                   break;
               case '*':
                   buffer[0] = '*';
                   break;
               case '/':
```

```
                buffer[0] = '/';
                break;
            case '^':
                buffer[0] = '^';
                break;
        }
        strcat(res,buffer);
        strcat(res,a);
        strcat(res,")");
        push(res);
        free(a);
        free(b);
    }
  }
  printf("%s",pop());
}

int main(){
  printf("Enter the postfix expression:\n");
  fgets(postfix, MAX, stdin);
  size_t len = strlen(postfix);
  if (len > 0 && postfix[len-1] == '\n') {
      postfix[len-1] = '\0';
  }
  printf("The infix expression is:\n");
  postfix_infix();
  printf("\n");
  return 0;
}
```

Output: -

```
Enter the postfix expression:
95+4*
The infix expression is:
((9+5)*4)
```

7. Write a program to implement three stacks in one array. Perform stack operations by taking the elements from a file. The selection of the stack is to be done randomly.

Code: -

```
#include<stdio.h>
#include<string.h>
#include<stdlib.h>
#include<time.h>
#define MAX_SIZE 99
#define STACKS 3
#define STACK_SIZE (MAX_SIZE/STACKS)

int stack[MAX_SIZE];
int tops[STACKS];

void constructStack(){
   for(int i=0;i<STACKS;++i){
       tops[i]=i*STACK_SIZE - 1;
   }
}

void push(int pos,int val){
   if(pos<0 || pos>=STACKS){
       printf("Invalid stack number\n");
       return;
   }
   if(tops[pos]>=(pos+1)*STACK_SIZE - 1){
       printf("Overflow\n");
       return;
   }
   stack[++tops[pos]]=val;
}

int pop(int pos){
   if(pos<0 || pos>=STACKS){
       printf("Invalid stack number\n");
```

```
        return -1;
    }
    if(tops[pos]<=pos*STACK_SIZE - 1){
        printf("Underflow\n");
        return -1;
    }
    return stack[tops[pos]--];
}

void printStack(){
    for(int i=0;i<STACKS;++i){
        if(tops[i] < i*STACK_SIZE)
            printf("Empty");
        else {
            for(int j = i*STACK_SIZE; j <= tops[i]; ++j)
                printf("%d ", stack[j]);
        }
        printf("\n");
    }
}

int main(){
    FILE *fp = fopen("input.txt", "r");
    if (fp == NULL) {
        printf("Failed to open input.txt\n");
        return 1;
    }
    char op[10];
    int val;
    constructStack();

    srand(time(0));

    while(fscanf(fp, "%s", op)!=EOF){
        int pos=rand()%STACKS;

        if(strcmp(op, "PUSH")==0){
            fscanf(fp, "%d",&val);
            push(pos, val);
```

```
        }
        else if(strcmp(op, "POP")==0){
            pop(pos);
        }
        else{
            printf("INVALID OPERATION: %s\n", op);
        }
    }
    printStack();
    fclose(fp);
}
```

Output: -

Assignment_1 > input.txt

```
1   PUSH 10
2   PUSH 20
3   PUSH 30
4   POP
5   PUSH 40
6   POP
7   POP
8   PUSH 50
9   PUSH 60
10  POP
11  PUSH 70
```

```
diptarko-bhattacharjee@diptarko:~/Desktop/Diptarko/Semester_Material/Sem_3/DSA/DSA_Lab$ cd Assignment_1
diptarko-bhattacharjee@diptarko:~/Desktop/Diptarko/Semester_Material/Sem_3/DSA/DSA_Lab/Assignment_1$ gcc Q7.c -o Q7
diptarko-bhattacharjee@diptarko:~/Desktop/Diptarko/Semester_Material/Sem_3/DSA/DSA_Lab/Assignment_1$ ./Q7 input.txt
Underflow
Underflow
10 60 70
Empty
30 40
diptarko-bhattacharjee@diptarko:~/Desktop/Diptarko/Semester_Material/Sem_3/DSA/DSA_Lab/Assignment_1$
```

8. Write the functions to implement queue using linked list. Take the elements from a file, which is to be given as the command line argument. Perform insertion and deletion randomly.

Code: -

```
#include<stdio.h>
#include<stdlib.h>

struct node {
   int data;
   struct node* next;
};

struct node* front = NULL;
struct node* rear = NULL;

int isempty(){
   if(front == NULL) return 1;
   else return 0;
}

void enqueue(int val){
   struct node* newnode = (struct node*)malloc(sizeof(struct node));
   newnode->data = val;
   newnode->next = NULL;
   if(rear == NULL){
       front = newnode;
       rear = newnode;
   }
   else{
       rear->next = newnode;
       rear = newnode;
   }
}

int dequeue(){
   if(isempty()){
       printf("No element in the queue");
```

```
        exit(1);
    }
    struct node* p = front;
    int val = p->data;
    front = front->next;
    free(p);
    return val;
}

void print(){
    if(isempty()){
        printf("No element in the queue\n");
        exit(1);
    }
    struct node* p = front;
    while(p != NULL){
        printf("%d ",p->data);
        p = p->next;
    }
    printf("\n");
}

void create(char* filename){
    FILE* fp = fopen(filename,"r");
    if(fp == NULL){
        printf("Error opening the file\n");
        return ;
    }

    int num;
    while(fscanf(fp,"%d",&num) != EOF){
        enqueue(num);
    }
}

int main(int argc, char* argv[]){
    printf("Hello!\n");
    if (argc != 2) {
        printf("Usage: %s <filename>\n", argv[0]);
```

```
        return 1;
    }
    create(argv[1]);
    printf("The queue is as follows: \n");
    print();

    while(1){
        int choice;
        printf("Enter your choice:\n");
        printf("Enter 1 for enqueue operation\n");
        printf("Enter 2 for dequeue operation\n");
        printf("Enter 3 to display the queue\n");
        printf("Enter 4 to exit\n");
        scanf("%d",&choice);

        if(choice == 4){
            break;
        }
        switch(choice){
            case 1:{
                printf("Enter the value:");
                int val;
                scanf("%d",&val);
                enqueue(val);
                break;
            }
            case 2: {
                int val = dequeue();
                printf("The value removed is %d\n",val);
                break;
            }
            case 3: {
                print();
                break;
            }
        }
    }
    return 0;
}
```

Output: -

Assignment_1 > Queue.txt

```
1  45 7 89 36 7 32 21 81
2
```

```
diptarko-bhattacharjee@diptarko:~/Desktop/Diptarko/Semester_Material/Sem_3/DSA/DSA_Lab/Assignment_1$ Q8.c -o Q8
Q8.c: command not found
diptarko-bhattacharjee@diptarko:~/Desktop/Diptarko/Semester_Material/Sem_3/DSA/DSA_Lab/Assignment_1$ gcc Q8.c -o Q8
diptarko-bhattacharjee@diptarko:~/Desktop/Diptarko/Semester_Material/Sem_3/DSA/DSA_Lab/Assignment_1$ ./Q8 Queue.txt
Hello!
The queue is as follows:
45 7 89 36 7 32 21 81
Enter your choice:
Enter 1 for enqueue operation
Enter 2 for dequeue operation
Enter 3 to display the queue
Enter 4 to exit
1
Enter the value:12
Enter your choice:
Enter 1 for enqueue operation
Enter 2 for dequeue operation
Enter 3 to display the queue
Enter 4 to exit
3
45 7 89 36 7 32 21 81 12
Enter your choice:
Enter 1 for enqueue operation
Enter 2 for dequeue operation
Enter 3 to display the queue
Enter 4 to exit
2
The value removed is 45
Enter your choice:
Enter 1 for enqueue operation
Enter 2 for dequeue operation
Enter 3 to display the queue
Enter 4 to exit
3
7 89 36 7 32 21 81 12
Enter your choice:
Enter 1 for enqueue operation
Enter 2 for dequeue operation
Enter 3 to display the queue
Enter 4 to exit
4
diptarko-bhattacharjee@diptarko:~/Desktop/Diptarko/Semester_Material/Sem_3/DSA/DSA_Lab/Assignment_1$
```

Assignment - 2:

1. Write a C program to build a complete binary tree using linked list. Take the elements from a file, which is to be given as the command line argument.

Code: -

```
#include<stdio.h>
#include<stdlib.h>

struct tree {
   struct tree* lchild;
   int data;
   struct tree* rchild;
};

struct node {
   struct tree* data;
   struct node* next;
};


struct tree *root;

struct node* front = NULL;
struct node* rear = NULL;

int isempty(){
   if(front == NULL) return 1;
   else return 0;
}

void enqueue(struct tree* val){
   struct node* newnode = (struct node*)malloc(sizeof(struct node));
   newnode->data = val;
   newnode->next = NULL;
   if(rear == NULL){
```

```
        // empty linked list no node is there
        front = newnode;
        rear = newnode;
    }
    else{
        rear->next = newnode;
        rear = newnode;
    }
}

struct tree* dequeue(){
    if(isempty()){
        printf("No element in the queue");
        exit(1);
    }
    struct node* p = front;
    struct tree* val = p->data;
    front = front->next;

    if(front == NULL){
        rear = NULL;
    }

    free(p);
    return val;
}


void create(char filename[]){
    FILE* fp = fopen(filename,"r");
    if(fp == NULL){
        printf("Error opening the file\n");
        return;
    }

    struct tree *p, *t;
    int x;
    if(fscanf(fp,"%d",&x) == 1){
        root = (struct tree*)malloc(sizeof(struct tree));
```

```
        root->data = x;
        root->lchild = NULL;
        root->rchild = NULL;
        enqueue(root);
    }
    else{
        return;
    }

    while(! isempty()){
        p = dequeue();
        if(fscanf(fp,"%d",&x) == 1){
            t = (struct tree*)malloc(sizeof(struct tree));
            t->data = x;
            t->lchild = NULL;
            t->rchild = NULL;
            p->lchild = t;
            enqueue(t);
        }
        else{
            return;
        }

        if(fscanf(fp,"%d",&x) == 1){
            t = (struct tree*)malloc(sizeof(struct tree));
            t->data = x;
            t->lchild = NULL;
            t->rchild = NULL;
            p->rchild = t;
            enqueue(t);
        }
        else{
            return;
        }
    }

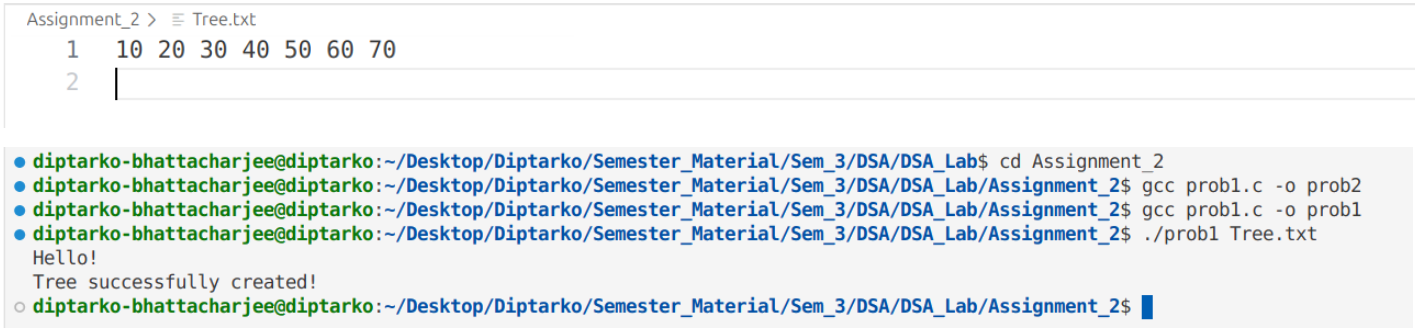
}
```

```
int main(int argc, char* argv[]){
   if (argc != 2) {
       printf("Usage: %s <filename>\n", argv[0]);
       return 1; // Indicate error
   }


   printf("Hello!\n");
   create(argv[1]);

   return 0;
}
```

Output: -



2. Write C programs for printing (i) inorder, (ii) postorder, and (iii) preorder traversals of the tree, generated by problem no. 1.

Code: -

```
#include<stdio.h>
#include<stdlib.h>

struct tree {
   struct tree* lchild;
   int data;
   struct tree* rchild;
};

struct node {
```

```
   struct tree* data;
   struct node* next;
};


struct tree *root;

struct node* front = NULL;
struct node* rear = NULL;

int isempty(){
   if(front == NULL) return 1;
   else return 0;
}

void enqueue(struct tree* val){
   struct node* newnode = (struct node*)malloc(sizeof(struct node));
   newnode->data = val;
   newnode->next = NULL;
   if(rear == NULL){
       front = newnode;
       rear = newnode;
   }
   else{
       rear->next = newnode;
       rear = newnode;
   }
}

struct tree* dequeue(){
   if(isempty()){
       printf("No element in the queue");
       exit(1);
   }
   struct node* p = front;
   struct tree* val = p->data;
   front = front->next;

   if(front == NULL){
```

```
        rear = NULL;
    }

    free(p);
    return val;
}


void create(char filename[]){
    FILE* fp = fopen(filename,"r");
    if(fp == NULL){
        printf("Error opening the file\n");
        return;
    }

    struct tree *p, *t;
    int x;
    if(fscanf(fp,"%d",&x) == 1){
        root = (struct tree*)malloc(sizeof(struct tree));
        root->data = x;
        root->lchild = NULL;
        root->rchild = NULL;
        enqueue(root);
    }
    else{
        return;
    }

    while(! isempty()){
        p = dequeue();
        if(fscanf(fp,"%d",&x) == 1){
            t = (struct tree*)malloc(sizeof(struct tree));
            t->data = x;
            t->lchild = NULL;
            t->rchild = NULL;
            p->lchild = t;
            enqueue(t);
        }
        else{
```

```
            return;
        }

        if(fscanf(fp,"%d",&x) == 1){
            t = (struct tree*)malloc(sizeof(struct tree));
            t->data = x;
            t->lchild = NULL;
            t->rchild = NULL;
            p->rchild = t;
            enqueue(t);
        }
        else{
            return;
        }
    }

}

void inorder(struct tree* p) {
    if (p) {
        inorder(p->lchild);
        printf("%d ", p->data);
        inorder(p->rchild);
    }
}

void preorder(struct tree* p) {
    if (p) {
        printf("%d ", p->data);
        preorder(p->lchild);
        preorder(p->rchild);
    }
}

void postorder(struct tree* p) {
    if (p) {
        postorder(p->lchild);
        postorder(p->rchild);
        printf("%d ", p->data);
```

```
    }
}

int main(int argc, char* argv[]){
    if (argc != 2) {
        printf("Usage: %s <filename>\n", argv[0]);
        return 1; // Indicate error
    }


    printf("Hello!\n");
    create(argv[1]);
    printf("Different types of tree traversal:\n");
    while(1){
        printf("\n 1. Preorder\n 2. Inorder \n 3. Postorder\n 4. Exit\n");
        printf("Enter the type of traversal:");
        int choice;
        scanf("%d",&choice);
        if(choice == 4){
            break;
        }

        switch(choice){
            case 1: preorder(root);
                    break;
            case 2: inorder(root);
                    break;
            case 3: postorder(root);
                    break;
        }
    }

    return 0;
}
```

Output: -

Assignment_2 > Tree.txt

```
1  10 20 30 40 50 60 70
2
```

```
diptarko-bhattacharjee@diptarko:~/Desktop/Diptarko/Semester_Material/Sem_3/DSA/DSA_Lab$ cd Assignment_2
diptarko-bhattacharjee@diptarko:~/Desktop/Diptarko/Semester_Material/Sem_3/DSA/DSA_Lab/Assignment_2$ gcc prob2.c -o prob2
diptarko-bhattacharjee@diptarko:~/Desktop/Diptarko/Semester_Material/Sem_3/DSA/DSA_Lab/Assignment_2$ ./prob2 Tree.txt
Hello!
Different types of tree traversal:

 1. Preorder
 2. Inorder
 3. Postorder
 4. Exit
Enter the type of traversal:1
10 20 40 50 30 60 70
 1. Preorder
 2. Inorder
 3. Postorder
 4. Exit
Enter the type of traversal:2
40 20 50 10 60 30 70
 1. Preorder
 2. Inorder
 3. Postorder
 4. Exit
Enter the type of traversal:3
40 50 20 60 70 30 10
 1. Preorder
 2. Inorder
 3. Postorder
 4. Exit
Enter the type of traversal:4
diptarko-bhattacharjee@diptarko:~/Desktop/Diptarko/Semester_Material/Sem_3/DSA/DSA_Lab/Assignment_2$
```

3. Write a C program to reconstruct the tree from its inorder and preorder traversals.

Code: -

```
#include <stdio.h>
#include <stdlib.h>

struct Node{
   int data;
   struct Node *left, *right;
};

struct Node* createNode(int data){
   struct Node *newNode = (struct Node*)malloc(sizeof(struct Node));
   newNode->data = data;
```

```
    newNode->left = newNode->right = NULL;
    return newNode;
}

int search(int inorder[], int start, int end, int value) {
    for (int i = start; i <= end; i++) {
        if (inorder[i] == value)
            return i;
    }
    return -1;
}

struct Node* buildTree(int inorder[], int preorder[], int inStart, int
inEnd, int *preIndex) {
    if (inStart > inEnd)
        return NULL;

    struct Node *root = createNode(preorder[*preIndex]);
    (*preIndex)++;

    if (inStart == inEnd)
        return root;

    int pos = search(inorder, inStart, inEnd, root->data);

    root->left = buildTree(inorder, preorder, inStart, pos - 1, preIndex);
    root->right = buildTree(inorder, preorder, pos + 1, inEnd, preIndex);
    return root;
}

void inorderTraversal(struct Node *root) {
  if (!root) return;
  inorderTraversal(root->left);
  printf("%d ", root->data);
  inorderTraversal(root->right);
}

void preorderTraversal(struct Node *root) {
  if (!root) return;
```

```
  printf("%d ", root->data);
  preorderTraversal(root->left);
  preorderTraversal(root->right);
}

void postorderTraversal(struct Node *root) {
  if (!root) return;
  postorderTraversal(root->left);
  postorderTraversal(root->right);
  printf("%d ", root->data);
}

int main() {
  int n;

  printf("Enter number of nodes: ");
  scanf("%d", &n);

  int inorder[n], preorder[n];

  printf("Enter inorder traversal:\n");
  for (int i = 0; i < n; i++)
      scanf("%d", &inorder[i]);

  printf("Enter preorder traversal:\n");
  for (int i = 0; i < n; i++)
      scanf("%d", &preorder[i]);

  int preIndex = 0;

  struct Node *root = buildTree(inorder, preorder, 0, n - 1, &preIndex);

  printf("\nTree Reconstructed Successfully.\n");

  printf("\nInorder traversal  : ");
  inorderTraversal(root);

  printf("\nPreorder traversal : ");
  preorderTraversal(root);
```

```
  printf("\nPostorder traversal: ");
  postorderTraversal(root);

  printf("\n");

  return 0;
}
```

Output: -

```
Enter number of nodes: 7
Enter inorder traversal:
13 27 18 17 15 2 24
Enter preorder traversal:
13 27 18 17 15 2 24

Tree Reconstructed Successfully.

Inorder traversal  : 13 27 18 17 15 2 24
Preorder traversal : 13 27 18 17 15 2 24
Postorder traversal: 24 2 15 17 18 27 13
diptarko-bhattacharjee@diptarko:~/Desktop/
```

4. Write a C program to build a max heap. Take the elements from a file, which is to be given as the command line argument.

Code: -

```
#include<stdio.h>
#include<stdlib.h>
#define MAX 100

void createHeap(int heap[],int n){
   int i = n;
   int temp = heap[n];
```

```
    while(i>1 && temp>heap[i/2]){
        heap[i] = heap[i/2];
        i = i/2;
    }

    heap[i] = temp;
}

int main(int argc, char* argv[]){
    printf("The max heap is as follows:\n");
    if(argc != 2){
        printf("Usage: %s <filename>\n", argv[0]);
        return 0;
    }

    FILE* fp = fopen(argv[1],"r");
    if(fp == NULL){
        printf("Error in opening file\n");
        return 0;
    }
    int num;
    int heap[MAX];
    heap[0] = 0;
    int size=1;

    while(fscanf(fp,"%d",&num) != EOF){
        heap[size] = num;
        size++;
    }

    for(int i=2;i<size;i++){
        createHeap(heap,i);
    }

    for(int i=0;i<size;i++){
        printf("%d  ",heap[i]);
    }
    return 0;
}
```

Output: -

Assignment_2 > Heap.txt

```
1  17 2 31 45 6 25 9 11 0
2
```

```
diptarko-bhattacharjee@diptarko:~/Desktop/Diptarko/Semester_Material/Sem_3/DSA/DSA_Lab$ cd Assignment_2
diptarko-bhattacharjee@diptarko:~/Desktop/Diptarko/Semester_Material/Sem_3/DSA/DSA_Lab/Assignment_2$ gcc prob4.c -o prob4
diptarko-bhattacharjee@diptarko:~/Desktop/Diptarko/Semester_Material/Sem_3/DSA/DSA_Lab/Assignment_2$ ./prob4 Heap.txt
The max heap is as follows:
0  45  31  25  11  6  17  9  2  0
diptarko-bhattacharjee@diptarko:~/Desktop/Diptarko/Semester_Material/Sem_3/DSA/DSA_Lab/Assignment_2$
```

5. Write a C program to build a min heap. Take the elements from a file, which is to be given as the command line argument.

Code: -

```
#include<stdio.h>
#include<stdlib.h>
#define MAX 100

void createHeap(int heap[],int n){
   int i = n;
   int temp = heap[n];

   while(i>1 && temp<heap[i/2]){
       heap[i] = heap[i/2];
       i = i/2;
   }
   heap[i] = temp;
}

int main(int argc, char* argv[]){
   printf("The min heap is as follows:\n");
   if(argc != 2){
       printf("Usage: %s <filename>\n", argv[0]);
       return 0;
   }

   FILE* fp = fopen(argv[1],"r");
```

```
    if(fp == NULL){
        printf("Error in opening file\n");
        return 0;
    }

    int num;
    int heap[MAX];
    heap[0] = 0;
    int size=1;

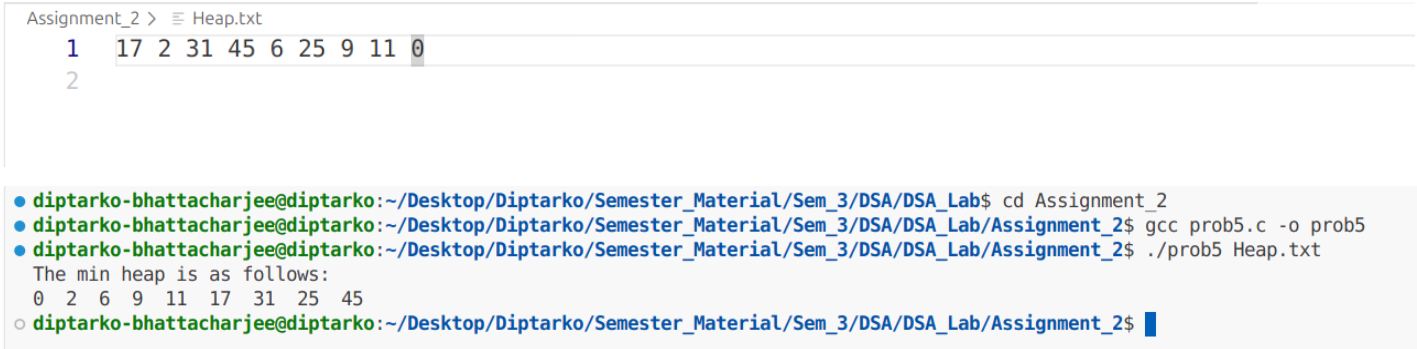
    while(fscanf(fp,"%d",&num) != EOF){
        heap[size] = num;
        size++;
    }

    // Function to create the heap
    for(int i=2;i<size;i++){
        createHeap(heap,i);
    }

    for(int i=0;i<size;i++){
        printf("%d  ",heap[i]);
    }

    return 0;
}
```

Output: -



6. Write insertion and deletion functions in C for a max heap represented as a lined binary tree. Assume that each node has a parent field as well as the usual left child, right child and data fields.

Code: -

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
   int data;
   struct Node *left, *right, *parent;
} Node;

Node* newNode(int val) {
   Node* n = (Node *)malloc(sizeof(Node));
   n->data = val;
   n->left = n->right = n->parent = NULL;
   return n;
}

typedef struct QueueNode {
   Node* tnode;
   struct QueueNode* next;
} QueueNode;

typedef struct Queue {
   QueueNode *front, *rear;
} Queue;

void enqueue(Queue* q, Node* t) {
   if (!t) return;
   QueueNode* temp = (QueueNode* )malloc(sizeof(QueueNode));
   temp->tnode = t;
   temp->next = NULL;
   if (!q->rear) q->front = q->rear = temp;
   else q->rear->next = temp, q->rear = temp;
}
```

```
Node* dequeue(Queue* q) {
   if (!q->front) return NULL;
   QueueNode* temp = q->front;
   Node* t = temp->tnode;
   q->front = q->front->next;
   if (!q->front) q->rear = NULL;
   free(temp);
   return t;
}

int empty(Queue* q) {
   return q->front == NULL;
}

void printLevelOrder(Node* root) {
   if (!root) return;
   Queue q = {0};
   enqueue(&q, root);
   printf("Heap (Level Order) traveral is as follows: ");
   while (!empty(&q)) {
       Node* t = dequeue(&q);
       printf("%d ", t->data);
       enqueue(&q, t->left);
       enqueue(&q, t->right);
   }
   printf("\n");
}

void heapify_up(Node* node) {
   while (node->parent && node->parent->data < node->data) {
       int t = node->data;
       node->data = node->parent->data;
       node->parent->data = t;
       node = node->parent;
   }
}

Node* insert(Node* root, int val) {
   Node* n = newNode(val);
```

```
    if (!root) return n;

    Queue q = {0};
    enqueue(&q, root);

    while (!empty(&q)) {
        Node* temp = dequeue(&q);

        if (!temp->left) {
            temp->left = n;
            n->parent = temp;
            heapify_up(n);
            return root;
        } else enqueue(&q, temp->left);

        if (!temp->right) {
            temp->right = n;
            n->parent = temp;
            heapify_up(n);
            return root;
        } else enqueue(&q, temp->right);
    }
    return root;
}

void heapify_down(Node* root) {
    while (root) {
        Node* largest = root;
        if (root->left && root->left->data > largest->data)
            largest = root->left;
        if (root->right && root->right->data > largest->data)
            largest = root->right;

        if (largest == root) break;

        int t = root->data;
        root->data = largest->data;
        largest->data = t;
```

```
        root = largest;
    }
}

Node* deleteMax(Node* root) {
    if (!root) return NULL;

    Queue q = {0};
    enqueue(&q, root);
    Node* last = NULL;
    while (!empty(&q)) {
        last = dequeue(&q);
        enqueue(&q, last->left);
        enqueue(&q, last->right);
    }

    if (last == root) {
        free(root);
        return NULL;
    }

    root->data = last->data;

    if (last->parent->left == last)
        last->parent->left = NULL;
    else
        last->parent->right = NULL;

    free(last);
    heapify_down(root);

    return root;
}

int main() {
    Node* root = NULL;

    printf("Inserting: 13\n");
    root = insert(root, 13);
```

```
    printLevelOrder(root);

    printf("Inserting: 27\n");
    root = insert(root, 27);
    printLevelOrder(root);

    printf("Inserting: 18\n");
    root = insert(root, 18);
    printLevelOrder(root);

    printf("Inserting: 17\n");
    root = insert(root, 17);
    printLevelOrder(root);

    printf("Inserting: 15\n");
    root = insert(root, 65);
    printLevelOrder(root);

    printf("Inserting: 2\n");
    root = insert(root, 2);
    printLevelOrder(root);

    printf("Inserting: 24\n");
    root = insert(root, 24);
    printLevelOrder(root);

    printf("Deleted the max element!\n");
    root = deleteMax(root);
    printLevelOrder(root);

    return 0;
}
```

Output: -

```
Inserting: 13
Heap (Level Order) traveral is as follows: 13
Inserting: 27
Heap (Level Order) traveral is as follows: 27 13
Inserting: 18
Heap (Level Order) traveral is as follows: 27 13 18
Inserting: 17
Heap (Level Order) traveral is as follows: 27 17 18 13
Inserting: 15
Heap (Level Order) traveral is as follows: 65 27 18 13 17
Inserting: 2
Heap (Level Order) traveral is as follows: 65 27 18 13 17 2
Inserting: 24
Heap (Level Order) traveral is as follows: 65 27 24 13 17 2 18
Deleted the max element!
Heap (Level Order) traveral is as follows: 27 18 24 13 17 2
diptarko-bhattacharjee@diptarko:~/Desktop/Diptarko/Semester Mate
```

7. Write a C program to build a binary search tree (BST) using linked list. Take the elements from a file, which is to be given as the command line argument.

Code: -

```
#include<stdio.h>
#include<stdlib.h>

struct node{
   struct node* lchild;
   int data;
   struct node* rchild;
};

struct node* root = NULL;

void insert(int key){
   struct node* t = root;
```

```
    struct node* r = NULL;
    struct node* p = NULL;

    if(root == NULL){
        p = (struct node*)malloc(sizeof(struct node));
        p->data = key;
        p->lchild = NULL;
        p->rchild = NULL;
        root = p;
        return;
    }

    while(t != NULL){
        r = t;
        if(key < t->data){
            t = t->lchild;
        }
        else if(key > t->data){
            t = t->rchild;
        }
        else return;
    }

    p = (struct node*)malloc(sizeof(struct node));
    p->data = key;
    p->lchild = NULL;
    p->rchild = NULL;

    if(key<r->data) r->lchild = p;
    else r->rchild = p;
}

void inorder(struct node* p) {
    if (p) {
        inorder(p->lchild);
        printf("%d ", p->data);
        inorder(p->rchild);
    }
}
```

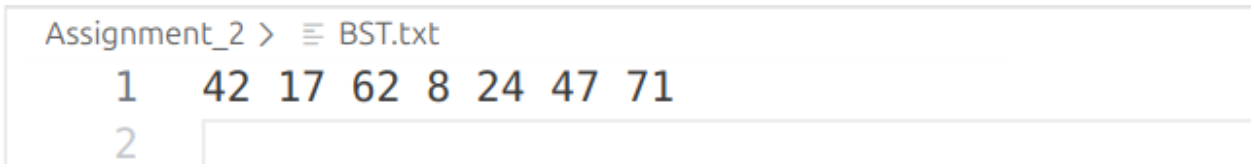
```
int main(int argc, char* argv[]){
   if (argc != 2) {
       printf("Usage: %s <filename>\n", argv[0]);
       return 1; // Indicate error
   }

   FILE* fp = fopen(argv[1],"r");
   if(fp == NULL){
       printf("Error opening the file\n");
       return 1;
   }

   int num;
   while(fscanf(fp,"%d",&num) == 1){
       insert(num);
   }
   fclose(fp);

   inorder(root);
   printf("\n");
   return 0;
}
```

Output: -




```
diptarko-bhattacharjee@diptarko:~/Desktop/Diptarko/Semester_Material/Sem_3/DSA/DSA_Lab$ cd Assignment_2
diptarko-bhattacharjee@diptarko:~/Desktop/Diptarko/Semester_Material/Sem_3/DSA/DSA_Lab/Assignment_2$ gcc prob7.c -o prob7
diptarko-bhattacharjee@diptarko:~/Desktop/Diptarko/Semester_Material/Sem_3/DSA/DSA_Lab/Assignment_2$ ./prob7 BST.txt
8 17 24 42 47 62 71 diptarko-bhattacharjee@diptarko:~/Desktop/Diptarko/Semester_Material/Sem_3/DSA/DSA_Lab/Assignment_2$
```

8. Write a C program to print nodes of a BST in descending order.

Code: -

```
#include<stdio.h>
#include<stdlib.h>

struct node{
   struct node* lchild;
   int data;
   struct node* rchild;
};

struct node* root = NULL;

void insert(int key){
   struct node* t = root;
   struct node* r = NULL;
   struct node* p = NULL;

   if(root == NULL){
       p = (struct node*)malloc(sizeof(struct node));
       p->data = key;
       p->lchild = NULL;
       p->rchild = NULL;
       root = p;
       return;
   }

   while(t != NULL){
       r = t;
       if(key < t->data){
           t = t->lchild;
       }
       else if(key > t->data){
           t = t->rchild;
       }
       else return;
   }
```

```
    p = (struct node*)malloc(sizeof(struct node));
    p->data = key;
    p->lchild = NULL;
    p->rchild = NULL;

    if(key<r->data) r->lchild = p;
    else r->rchild = p;
}

void printdesc(struct node* p) {
    if (p) {
        printdesc(p->rchild);
        printf("%d ", p->data);
        printdesc(p->lchild);
    }
}

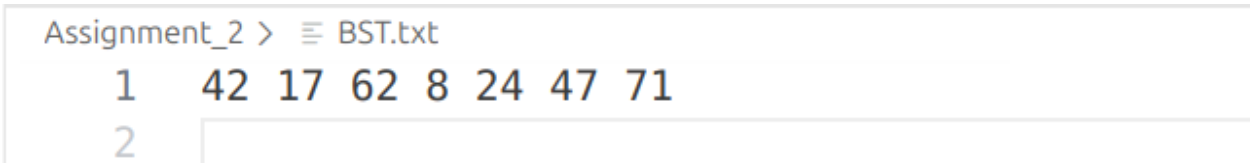
int main(int argc, char* argv[]){
    if (argc != 2) {
        printf("Usage: %s <filename>\n", argv[0]);
        return 1; // Indicate error
    }

    FILE* fp = fopen(argv[1],"r");
    if(fp == NULL){
        printf("Error opening the file\n");
        return 1;
    }

    int num;
    while(fscanf(fp,"%d",&num) == 1){
        insert(num);
    }
    fclose(fp);

    printdesc(root);
    return 0;
}
```

Output: -




```
diptarko-bhattacharjee@diptarko:~/Desktop/Diptarko/Semester_Material/Sem_3/DSA/DSA_Lab$ cd Assignment_2
diptarko-bhattacharjee@diptarko:~/Desktop/Diptarko/Semester_Material/Sem_3/DSA/DSA_Lab/Assignment_2$ gcc prob8.c -o prob8
diptarko-bhattacharjee@diptarko:~/Desktop/Diptarko/Semester_Material/Sem_3/DSA/DSA_Lab/Assignment_2$ ./prob8 BST.txt
71 62 47 42 24 17 8 diptarko-bhattacharjee@diptarko:~/Desktop/Diptarko/Semester_Material/Sem_3/DSA/DSA_Lab/Assignment_2$
```

9. Write a C program to build an AVL tree using linked list. Take the elements from a file, which is to be given as the command line argument.

Code: -

```
#include<stdio.h>
#include<stdlib.h>

struct node{
   struct node* lchild;
   int data;
   int height;
   struct node* rchild;
};
struct node* root = NULL;

int nodeHeight(struct node* p){
   int hl,hr;
   if(p && p->lchild){
       hl = p->lchild->height;
   }
   else hl = 0;

   if(p && p->rchild){
       hr = p->rchild->height;
   }
   else hr = 0;

   if(hl > hr) return hl+1;
```

```
   else return hr+1;
}

int balanceFactor(struct node* p){
   int hl,hr;
   if(p && p->lchild){
       hl = p->lchild->height;
   }
   else hl = 0;
   if(p && p->rchild){
       hr = p->rchild->height;
   }
   else hr = 0;

   return hl-hr;
}

struct node* rotationLL(struct node* p){
   struct node* pl = p->lchild;
   struct node* plr = pl->rchild;

   pl->rchild = p;
   p->lchild = plr;

   p->height = nodeHeight(p);
   pl->height = nodeHeight(pl);

   if(root == p){
       root = pl;
   }
   return pl;
}

struct node* rotationRR(struct node* p){
   struct node* pr = p->rchild;
   struct node* prl = pr->lchild;

   pr->lchild = p;
   p->rchild = prl;
```

```
    p->height = nodeHeight(p);
    pr->height = nodeHeight(pr);

    if(root == p){
        root = pr;
    }
    return pr;
}

struct node* rotationLR(struct node* p){
    struct node* pl = p->lchild;
    struct node* plr = pl->rchild;

    pl->rchild = plr->lchild;
    p->lchild = plr->rchild;
    plr->rchild = p;
    plr->lchild = pl;

    p->height = nodeHeight(p);
    pl->height = nodeHeight(pl);
    plr->height = nodeHeight(plr);

    if(root == p){
        root = plr;
    }
    return plr;
}

struct node* rotationRL(struct node* p){
    struct node* pr = p->rchild;
    struct node* prl = pr->lchild;

    p->rchild = prl->lchild;
    pr->lchild = prl->rchild;
    prl->lchild = p;
    prl->rchild = pr;

    p->height = nodeHeight(p);
```

```
   pr->height = nodeHeight(pr);
   prl->height = nodeHeight(prl);

   if(root == p){
       root = prl;
   }
   return prl;
}

struct node* insert(struct node* p,int key){
   struct node* temp;
   if(p==NULL){
       temp = (struct node*)malloc(sizeof(struct node));
       temp->data = key;
       temp->lchild = NULL;
       temp->rchild = NULL;
       temp->height = 1;
       return temp;
   }

   if(key < p->data){
       p->lchild = insert(p->lchild,key);
   }
   else if(key > p->data){
       p->rchild = insert(p->rchild,key);
   }
   p->height = nodeHeight(p);

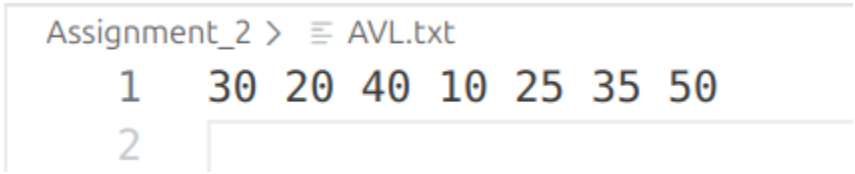
   if(balanceFactor(p) == 2 && balanceFactor(p->lchild) == 1) return
rotationLL(p);
   else if(balanceFactor(p) == 2 && balanceFactor(p->lchild) == -1) return
rotationLR(p);
   else if(balanceFactor(p) == -2 && balanceFactor(p->rchild) == -1) return
rotationRR(p);
   else if(balanceFactor(p) == -2 && balanceFactor(p->rchild) == 1) return
rotationRL(p);
   return p;

}
```

```
void inorder(struct node* root){
   if(root != NULL){
       inorder(root->lchild);
       printf("%d ",root->data);
       inorder(root->rchild);
   }
}

int main(int argc, char* argv[]){
   printf("Hello\n");
   if (argc != 2) {
       return 1; // Indicate error
   }
   FILE* fp = fopen(argv[1],"r");
   if(fp == NULL){
       printf("Error in opening the file\n");
       return 0;
   }
   int num;
   while(fscanf(fp,"%d",&num) != EOF){
       root = insert(root,num);
   }
   printf("The value of the tree inorder: ");
   inorder(root);
   return 0;
}
```

Output: -




```
diptarko-bhattacharjee@diptarko:~/Desktop/Diptarko/Semester_Material/Sem_3/DSA/DSA_Lab$ cd Assignment_2
diptarko-bhattacharjee@diptarko:~/Desktop/Diptarko/Semester_Material/Sem_3/DSA/DSA_Lab/Assignment_2$ gcc prob9.c -o prob9
diptarko-bhattacharjee@diptarko:~/Desktop/Diptarko/Semester_Material/Sem_3/DSA/DSA_Lab/Assignment_2$ ./prob9 AVL.txt
Hello
The value of the tree inorder: 10 20 25 30 35 40 50 diptarko-bhattacharjee@diptarko:~/Desktop/Diptarko/Semester_Material/Sem_3/
$
```

10. Write a C program to delete a node in an AVL tree, as generated in problem no. 9.

Code: -

```
#include<stdio.h>
#include<stdlib.h>

struct node{
   struct node* lchild;
   int data;
   struct node* rchild;
};

struct node* root = NULL;

void insert(int key){
   struct node* t = root;
   struct node* r = NULL;
   struct node* p = NULL;
   if(root == NULL){
       p = (struct node*)malloc(sizeof(struct node));
       p->data = key;
       p->lchild = NULL;
       p->rchild = NULL;
       root = p;
       return;
   }

   while(t != NULL){
       r = t;
       if(key < t->data){
           t = t->lchild;
       }
       else if(key > t->data){
           t = t->rchild;
       }
       else return;
```

```
    }

    p = (struct node*)malloc(sizeof(struct node));
    p->data = key;
    p->lchild = NULL;
    p->rchild = NULL;
    if(key<r->data) r->lchild = p;
    else r->rchild = p;
}

void inorder(struct node* p) {
    if (p) {
        inorder(p->lchild);
        printf("%d ", p->data);
        inorder(p->rchild);
    }
}

int height(struct node* p){
    if(p==NULL) return 0;
    int x = height(p->lchild);
    int y = height(p->rchild);
    if(x>y) return x+1;
    else return y+1;
}

struct node* inPre(struct node* p){
    while(p && p->rchild){
        p = p->rchild;
    }
    return p;
}

struct node* inSucc(struct node* p){
    while(p && p->lchild){
        p = p->lchild;
    }
    return p;
}
```

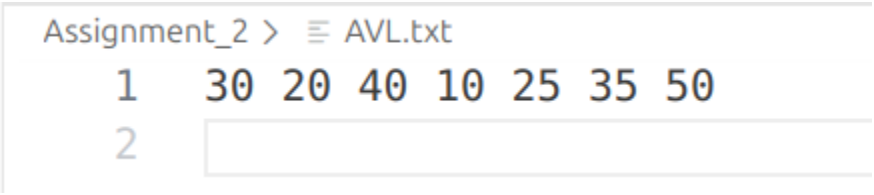
```
struct node* delete(struct node* p,int key){
   if(p==NULL) return NULL;
   if(p->lchild == NULL && p->rchild == NULL){
       if(p==root) root = NULL;
       free(p);
       return NULL;
   }

   if(key < p->data){
       p->lchild = delete(p->lchild,key);
   }
   else if(key > p->data){
       p->rchild = delete(p->rchild,key);
   }
   else{
       struct node* q = NULL;
       if(height(p->lchild) > height(p->rchild)){
           q = inPre(p->lchild);
           p->data = q->data;
           p->lchild = delete(p->lchild,q->data);
       }
       else{
           q = inSucc(p->rchild);
           p->data = q->data;
           p->rchild = delete(p->rchild,q->data);
       }
   }
   return p;
}

int main(int argc, char* argv[]){
   if (argc != 2) {
       return 1; // Indicate error
   }
   FILE* fp = fopen(argv[1],"r");
   if(fp == NULL){
       printf("Error opening the file\n");
       return 1;
   }
```

```
    int num;
    while(fscanf(fp,"%d",&num) == 1){
        insert(num);
    }
    fclose(fp);
    inorder(root);
    root = delete(root,20);
    printf("\n");
    inorder(root);
    return 0;
}
```

Output: -




```
diptarko-bhattacharjee@diptarko:~/Desktop/Diptarko/Semester_Material/Sem_3/DSA/DSA_Lab$ cd Assignment_2
diptarko-bhattacharjee@diptarko:~/Desktop/Diptarko/Semester_Material/Sem_3/DSA/DSA_Lab/Assignment_2$ gcc prob10.c -o prob10
diptarko-bhattacharjee@diptarko:~/Desktop/Diptarko/Semester_Material/Sem_3/DSA/DSA_Lab/Assignment_2$ ./prob10 AVL.txt
Before deletion:
The value of the tree inorder: 10 20 25 30 35 40 50
Enter value to delete: 35
After deletion:
The value of the tree inorder: 10 20 25 30 40 50
diptarko-bhattacharjee@diptarko:~/Desktop/Diptarko/Semester_Material/Sem_3/DSA/DSA_Lab/Assignment_2$
```

Assignment - 3:

1. Using quick sort algorithm, write C programs to determine the number of comparisons and swap operations when the pivot is chosen as (i) the first element, (ii) random element, and (iii) median-of-three. Take the elements as the digits of your mobile number and stored in a file, which is to be given as the command line argument.

Code: -

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

long long compCount = 0;
long long swapCount = 0;

void swap(int *a, int *b) {
   swapCount++;
   int temp = *a;
   *a = *b;
   *b = temp;
}

int partition_first(int arr[], int low, int high) {
   int pivot = arr[low];
   int i = low + 1;

   for(int j = low + 1; j <= high; j++) {
       compCount++;
       if (arr[j] < pivot) {
           swap(&arr[i], &arr[j]);
           i++;
       }
   }
   swap(&arr[low], &arr[i - 1]);
   return i - 1;
}
```

```
int partition_random(int arr[], int low, int high) {
   int randomIndex = low + rand() % (high - low + 1);
   swap(&arr[low], &arr[randomIndex]);
   return partition_first(arr, low, high);
}

int median_of_three(int a, int b, int c) {
   if ((a > b) ^ (a > c)) return a;
   if ((b > a) ^ (b > c)) return b;
   return c;
}

int find_median_index(int arr[], int low, int high) {
   int mid = (low + high) / 2;
   int a = arr[low], b = arr[mid], c = arr[high];
   int median = median_of_three(a, b, c);

   if (median == a) return low;
   if (median == b) return mid;
   return high;
}

int partition_median(int arr[], int low, int high) {
   int medianIndex = find_median_index(arr, low, high);
   swap(&arr[low], &arr[medianIndex]); // move median pivot to first
   return partition_first(arr, low, high);
}


void quicksort(int arr[], int low, int high, int mode) {
   if (low < high) {
       int p;

       if (mode == 1)
           p = partition_first(arr, low, high);
       else if (mode == 2)
           p = partition_random(arr, low, high);
       else
           p = partition_median(arr, low, high);
```

```
        quicksort(arr, low, p - 1, mode);
        quicksort(arr, p + 1, high, mode);
    }
}

int main(int argc, char *argv[]) {
    if (argc != 2) {
        printf("Usage: %s <inputfile>\n", argv[0]);
        return 1;
    }

    FILE *fp = fopen(argv[1], "r");
    if (!fp) {
        printf("Cannot open file %s\n", argv[1]);
        return 1;
    }

    int arr[100], n = 0;

    while (fscanf(fp, "%d", &arr[n]) == 1)
        n++;

    fclose(fp);

    int a1[100], a2[100], a3[100];

    for (int i = 0; i < n; i++) {
        a1[i] = arr[i];
        a2[i] = arr[i];
        a3[i] = arr[i];
    }

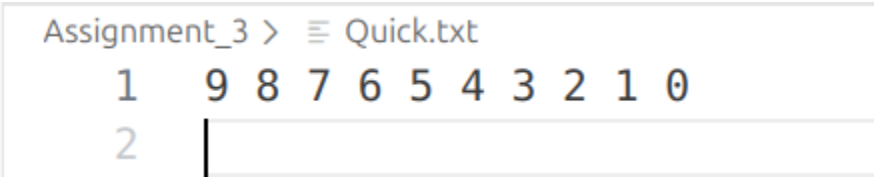

    compCount = swapCount = 0;
    quicksort(a1, 0, n - 1, 1);
    printf("\nPivot = First Element:\n");
    printf("Comparisons: %lld\nSwaps: %lld\n", compCount, swapCount);
```

```
    compCount = swapCount = 0;
    quicksort(a2, 0, n - 1, 2);
    printf("\nPivot = Random Element:\n");
    printf("Comparisons: %lld\nSwaps: %lld\n", compCount, swapCount);

    compCount = swapCount = 0;
    quicksort(a3, 0, n - 1, 3);
    printf("\nPivot = Median-of-Three:\n");
    printf("Comparisons: %lld\nSwaps: %lld\n", compCount, swapCount);

    return 0;
}
```

Output: -




```
diptarko-bhattacharjee@diptarko:~/Desktop/Diptarko/Semester_Material/Sem_3/DSA/DSA_Lab$ cd Assignment_3
diptarko-bhattacharjee@diptarko:~/Desktop/Diptarko/Semester_Material/Sem_3/DSA/DSA_Lab/Assignment_3$ gcc q1.c -o q1
diptarko-bhattacharjee@diptarko:~/Desktop/Diptarko/Semester_Material/Sem_3/DSA/DSA_Lab/Assignment_3$ ./q1 Quick.txt
Pivot = First Element:
Comparisons: 45
Swaps: 34

Pivot = Random Element:
Comparisons: 21
Swaps: 24

Pivot = Median-of-Three:
Comparisons: 19
Swaps: 19
diptarko-bhattacharjee@diptarko:~/Desktop/Diptarko/Semester_Material/Sem_3/DSA/DSA_Lab/Assignment_3$
```

2. Write C programs for heap sort using the input 26, 5, 77, 1, 61, 11, 59, 15, 48, 19, which are stored in a file. Determine the number of swap operations performed for the same.

Code: -

```
#include <stdio.h>
#include <stdlib.h>
```

```
long long swapcnt = 0;

void swap(int *x, int *y) {
   int temp = *x;
   *x = *y;
   *y = temp;
   swapcnt++;
}

void heapify(int arr[], int n, int i) {
   int largest = i;
   int left = 2*i +1;
   int right = 2*i +2;
   if (left < n && arr[left] > arr[largest])
       largest = left;
   if (right < n && arr[right] > arr[largest])
       largest = right;
   if (largest != i) {
       swap(&arr[i], &arr[largest]);
       heapify(arr, n, largest);
   }
}

void heapSort(int arr[], int n) {
   for (int i = n / 2 - 1; i >= 0; i--)
       heapify(arr, n, i);
   for (int i = n - 1; i >= 0; i--) {
       swap(&arr[0], &arr[i]);
       heapify(arr, i, 0);
   }
}

int main(int argc, char *argv[]) {
   FILE *fp = fopen(argv[1], "r");
   if (!fp) {
       printf("Cannot open file %s!\n", argv[1]);
       return 1;
   }
```

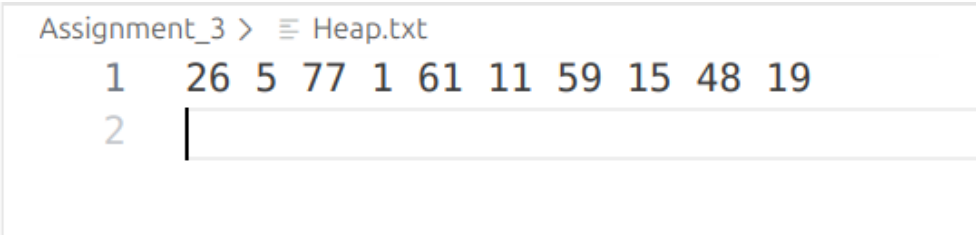
```
    int arr[100], n = 0;
    while(fscanf(fp, "%d", &arr[n]) == 1)
        n++;
    fclose(fp);

    swapcnt = 0;
    heapSort(arr, n);
    printf("Sorted output is as follows:\n");
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);

    printf("\nTotal swap operations = %lld\n", swapcnt);

    return 0;
}
```

Output: -




```
diptarko-bhattacharjee@diptarko:~/Desktop/Diptarko/Semester_Material/Sem_3/DSA/DSA_Lab$ cd Assignment_3
diptarko-bhattacharjee@diptarko:~/Desktop/Diptarko/Semester_Material/Sem_3/DSA/DSA_Lab/Assignment_3$ gcc q2.c -o q2
diptarko-bhattacharjee@diptarko:~/Desktop/Diptarko/Semester_Material/Sem_3/DSA/DSA_Lab/Assignment_3$ ./q2 Heap.txt
Sorted output is as follows:
1 5 11 15 19 26 48 59 61 77
Total swap operations = 29
diptarko-bhattacharjee@diptarko:~/Desktop/Diptarko/Semester_Material/Sem_3/DSA/DSA_Lab/Assignment_3$
```

3. Write a C program for merge sort using the elements given in problem 2.

Code: -

```
#include <stdio.h>
#include <stdlib.h>

void merge(int arr[], int l, int mid, int r) {
    int n1 = mid - l + 1;
```

```
    int n2 = r - mid;

    int *L = (int *)malloc(n1 * sizeof(int));
    int *R = (int *)malloc(n2 * sizeof(int));

    for(int i = 0; i < n1; i++)
        L[i] = arr[l + i];

    for(int i = 0; i < n2; i++)
        R[i] = arr[mid + 1 + i];

    int i = 0, j = 0, k = l;

    while (i < n1 && j < n2) {
        if (L[i] <= R[j])
            arr[k++] = L[i++];
        else
            arr[k++] = R[j++];
    }

    while (i < n1)
        arr[k++] = L[i++];

    while (j < n2)
        arr[k++] = R[j++];

    free(L);
    free(R);
}

void mergeSort(int arr[], int l, int r) {
    if (l < r) {
        int mid = (l + r) / 2;
        mergeSort(arr, l, mid);
        mergeSort(arr, mid + 1, r);
        merge(arr, l, mid, r);
    }
}
```

```
int main(int argc, char *argv[]) {

   FILE *fp = fopen(argv[1], "r");
   if (!fp) {
       printf("Error opening file.\n");
       return 1;
   }

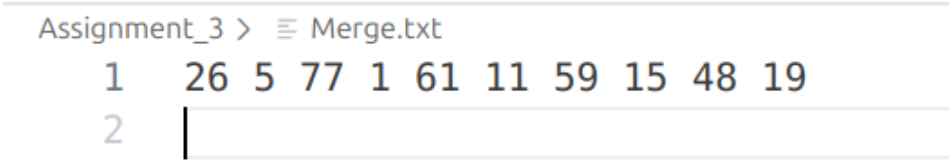
   int arr[1000], n = 0;

   while(fscanf(fp, "%d", &arr[n]) != EOF)
       n++;

   fclose(fp);

   printf("Original array:\n");
   for (int i = 0; i < n; i++)
       printf("%d ", arr[i]);
   printf("\n");
   mergeSort(arr, 0, n - 1);
   printf("\nSorted array:\n");
   for (int i = 0; i < n; i++)
       printf("%d ", arr[i]);
   printf("\n");
   return 0;
}
```

Output: -




```
diptarko-bhattacharjee@diptarko:~/Desktop/Diptarko/Semester_Material/Sem_3/DSA/DSA_Lab$ cd Assignment_3
diptarko-bhattacharjee@diptarko:~/Desktop/Diptarko/Semester_Material/Sem_3/DSA/DSA_Lab/Assignment_3$ gcc q3.c -o q3
diptarko-bhattacharjee@diptarko:~/Desktop/Diptarko/Semester_Material/Sem_3/DSA/DSA_Lab/Assignment_3$ ./q3 Merge.txt
Original array:
26 5 77 1 61 11 59 15 48 19

Sorted array:
1 5 11 15 19 26 48 59 61 77
diptarko-bhattacharjee@diptarko:~/Desktop/Diptarko/Semester_Material/Sem_3/DSA/DSA_Lab/Assignment_3$
```

4. Given input {5271, 1234, 6754, 4199, 6979, 1999, 2352} in a file and hash function h(x) = x mod 10. Show the results using (i) separate chaining hash table, (ii) open addressing hash table using linear probing, (iii) open addressing hash table using quadratic probing and (iv) open addressing hash table with second hash function h2(x) = 7 – (x mod 7).

Code: -

```
#include <stdio.h>
#include <stdlib.h>
#define TABLE_SIZE 10

typedef struct node {
   int key;
   struct node *next;
} Node;

Node *chain[TABLE_SIZE];

void insert_chaining(int key) {
   int index = key % TABLE_SIZE;
   Node *newNode = (Node *)malloc(sizeof(Node));
   newNode->key = key;
   newNode->next = chain[index];
   chain[index] = newNode;
}

void display_chaining() {
   printf("\nSeparate Chaining:\n");
   for (int i = 0; i < TABLE_SIZE; i++) {
       printf("%d --> ", i);
       Node *temp = chain[i];
       while (temp) {
           printf("%d -> ", temp->key);
           temp = temp->next;
       }
       printf("NULL\n");
   }
}
```

```
int linear[TABLE_SIZE];

void insert_linear(int key) {
   int index = key % TABLE_SIZE;
   int i = 0;

   while (linear[(index + i) % TABLE_SIZE] != -1)
       i++;

   linear[(index + i) % TABLE_SIZE] = key;
}

void display_linear() {
   printf("\nLinear Probing:\n");
   for (int i = 0; i < TABLE_SIZE; i++)
       printf("%d : %d\n", i, linear[i]);
}

int quadratic[TABLE_SIZE];

void insert_quadratic(int key) {
   int index = key % TABLE_SIZE;
   int i = 0;

   while(quadratic[(index + i*i) % TABLE_SIZE] != -1)
       i++;

   quadratic[(index + i*i) % TABLE_SIZE] = key;
}

void display_quadratic() {
   printf("\nQuadratic Probing:\n");
   for (int i = 0; i < TABLE_SIZE; i++)
       printf("%d : %d\n", i, quadratic[i]);
}

int dbl[TABLE_SIZE];

int h2(int x) {
```

```
    return 7 - (x % 7);
}

void insert_double(int key) {
    int index = key % TABLE_SIZE;
    int step = h2(key);
    int i = 0;

    while (dbl[(index + i * step) % TABLE_SIZE] != -1)
        i++;

    dbl[(index + i * step) % TABLE_SIZE] = key;
}

void display_double() {
    printf("\nDouble Hashing:\n");
    for (int i = 0; i < TABLE_SIZE; i++)
        printf("%d : %d\n", i, dbl[i]);
}


int main(int argc, char *argv[]) {
    FILE *fp = fopen(argv[1], "r");
    if (!fp) {
        printf("Error: Cannot open file.\n");
        return 1;
    }

    int keys[100], n = 0;
    while (fscanf(fp, "%d", &keys[n]) != EOF)
        n++;
    fclose(fp);

    for (int i = 0; i < TABLE_SIZE; i++) {
        chain[i] = NULL;
        linear[i] = -1;
        quadratic[i] = -1;
        dbl[i] = -1;
    }
```

```
    for (int i = 0; i < n; i++) {
        insert_chaining(keys[i]);
        insert_linear(keys[i]);
        insert_quadratic(keys[i]);
        insert_double(keys[i]);
    }

    display_chaining();
    display_linear();
    display_quadratic();
    display_double();

    return 0;
}
```

Output: -

Assignment_3 > Hash.txt

```
1  5271 1234 6754 4199 6979 1999 2352
2
```

```
diptarko-bhattacharjee@diptarko:~/Desktop/Diptarko/Semester_Material/Sem_3/DSA/DSA_Lab$ cd Assignment_3
diptarko-bhattacharjee@diptarko:~/Desktop/Diptarko/Semester_Material/Sem_3/DSA/DSA_Lab/Assignment_3$ gcc q4.c -o q4
diptarko-bhattacharjee@diptarko:~/Desktop/Diptarko/Semester_Material/Sem_3/DSA/DSA_Lab/Assignment_3$ ./q4 Hash.txt

Separate Chaining:
0 --> NULL
1 --> 5271 -> NULL
2 --> 2352 -> NULL
3 --> NULL
4 --> 6754 -> 1234 -> NULL
5 --> NULL
6 --> NULL
7 --> NULL
8 --> NULL
9 --> 1999 -> 6979 -> 4199 -> NULL

Linear Probing:
0 : 6979
1 : 5271
2 : 1999
3 : 2352
4 : 1234
5 : 6754
6 : -1
7 : -1
8 : -1
9 : 4199

Quadratic Probing:
0 : 6979
1 : 5271
2 : 2352
3 : 1999
4 : 1234
5 : 6754
6 : -1
7 : -1
8 : -1
9 : 4199

Double Hashing:
0 : -1
1 : 5271
2 : 1999
3 : 2352
4 : 1234
5 : 6754
6 : 6979
7 : -1
8 : -1
9 : 4199
diptarko-bhattacharjee@diptarko:~/Desktop/Diptarko/Semester_Material/Sem_3/DSA/DSA_Lab/Assignment_3$
```